Target 2026!! To be the best developer & go to big tech.

NODE.JS

Date 09/02/2026.

* Node.Js is a Javascript runtime & in top of it built on chrome V8 engine.

* Node.Js has an "event-driven architechture" & capable of asynchronous I/o.

* whereever Javascript runs Javascript has an engine.
  => spidermonkey (Firefox Js engine)
  => V8 (Google chrome Js engine).

Node Js - Js on server

* All the Js engines should follow ECMA Script standard.

* In frontend -> Global object refers to window & this, self, frames.
* In Node Js -> Global object refers only global & when 2020 the open Js community declares that globalThis will be the common global object across Everywhere Js/NodeJs/chrome/firefox & every where.

Date 10/02/2026.

* If you need to run one module into another module you need to give require ("filename"). So it will consider that file also to execute in the terminal. you can only the access one file into another file if the file doesn't have any functions, variables & methods. If the file has a functions, methods and variables you cannot use "require" only to get / run the access/file.

MORE ABOUT MODULE EXPORTS::

There are several ways to exports the file.

* we export the file to just ex. save the private variables, functions & methods. If we only do module exports we are only giving the access to use it in another file.

* There are two types of modules in node JS.

They are:: ① common JS Module
② ES Module.

Date 10/02/2026

Common JS modules (CJS)	ES JS Modules. (MJS)
→ Exporting: module.exports	→ Import
→ Importing: require ( )	→ Exports.
⇒ runs as non strict mode.	⇒ Strict mode.
→ By default used in node	→ By default used in React, Angular.
⇒ older way	⇒ newer way.
⇒ only synchronous	⇒ Asynchronous

* By default vs code will take common JS to execute node JS files. But if you want to use the ES modules. You need to ↓

Package.Json :: {

type: "module"

}

By adding this in package.Json it will take as ES JS module.

In ES module case you can do " z = "Hello" " ; without any "var, const & let" but in CJS you need to include using ↑ this

Date 10/02/2026

* console.log(module.exports) = { }

So for Example

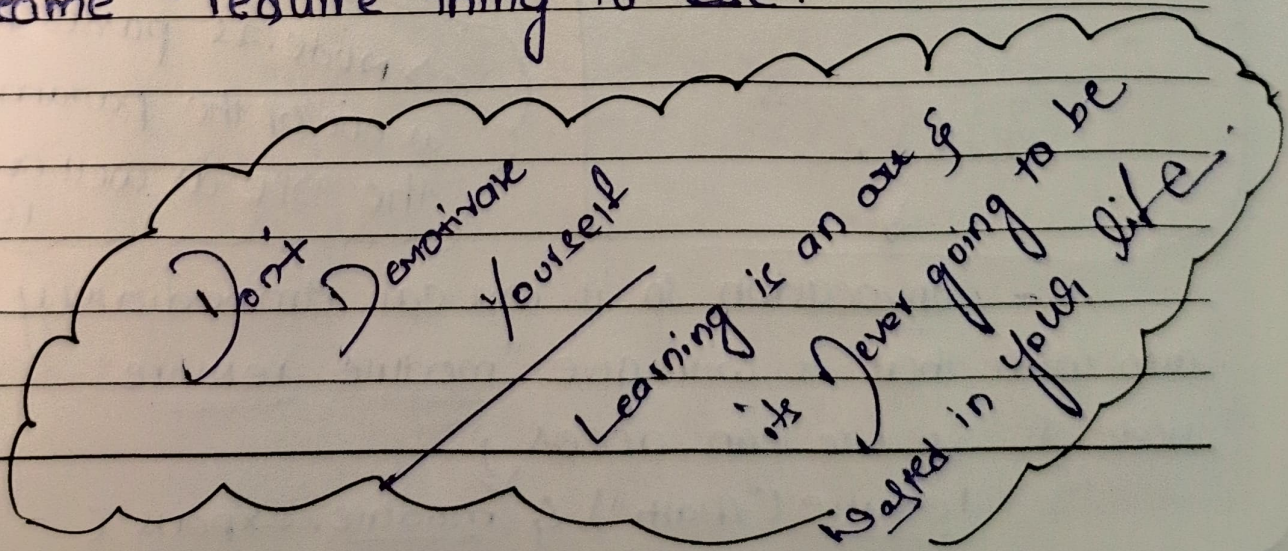
① module.exports { x : x ,
abc : abc } ;

② module.exports {x, abc } ;

③ module.exports.x = x ;
module.exports.abc = abc ;

All the above 3 module.exports are same. you can follow however you want.

* suppose you need Json files to import. you need the same require thing to use.



Interview Question:

Important concept:

Date 10/02/2026

=> Why you cannot use directly one module into another & module without module.exports & require("/path") to import.

Because, when ever you use require("/path") the node.js puts all the module inside the function which is IIFE (Immediately Invoked function Expression) So because of this we cannot access the functions, variabus & methods because of the concept "SCOPE CHAIN"

HOW IIFE WORKS?

whenever you use IIFE, it creates a Immediate invoking function for the whole module. How does the function looks like?

```
(function () {

})();
```

** please note that it

=> Node.js passes module as one of the parameter to the IIFE as well as require

Invocation, so it will call Immediately & also with that it will give "module, require" by the node.js so we can access

require("/path") & module.exports ( )

Date 10/02/2026.

* what exactly require("/path") do?

① Resolving the module
   => ./localpath
   => .Json
   => node_module.

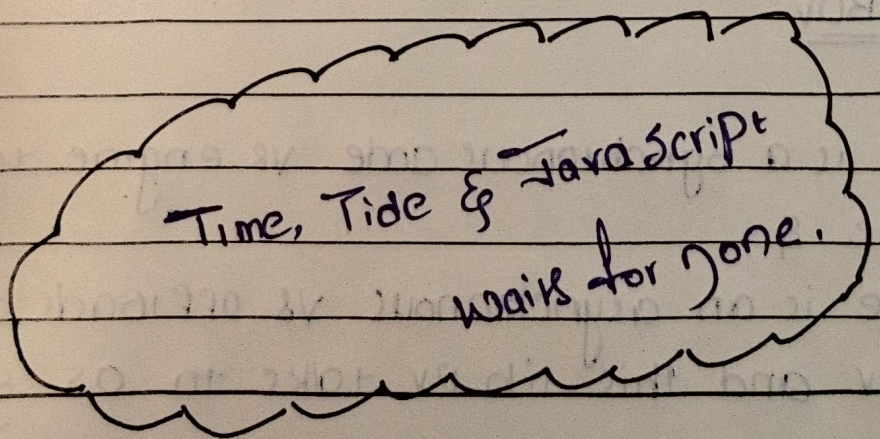
② Loading the module
   => File content is loaded
   => Account to type file.

③ wraps inside IIFE

④ Evaluation.
   => module.exports

⑤ caching.

For example:. A single file is reused once again in the multiple files it will be read only once & taken from the cache. node.js won't do all the 4 steps again. It will directly taken from the cache.



Date 11 / 02 / 2026.

* Javascript Engine doesn't have any timers/nothing to execute the code after sometime.

LIBUV:- Since node.js runs in the synchronous task if you want to do the asynchronous tasks "libuv" take the charge and do whatever the asynchronous task you want!

Eg:- * Getting the file from the local drive of OS.
* contacting to the DB.
* Making API calls.
* Doing settimeout.

Simple words:- "LIBUV" acts a middle layer between your V8 engine and the operating system. Without libuv you cannot make async operations.

one of the important :- "NODE.JS" is asynchronous because of "LIBUV"

⇒ when there is a synchronous code V8 engine takes care the code &
⇒ when there is an asynchronous V8 offloads the code to libuv and this lib uv takes to OS & Execute.

LIBUV HELPS TO RUN ASYNC CODE

Date 11 / 02/2026

Garbage collection:

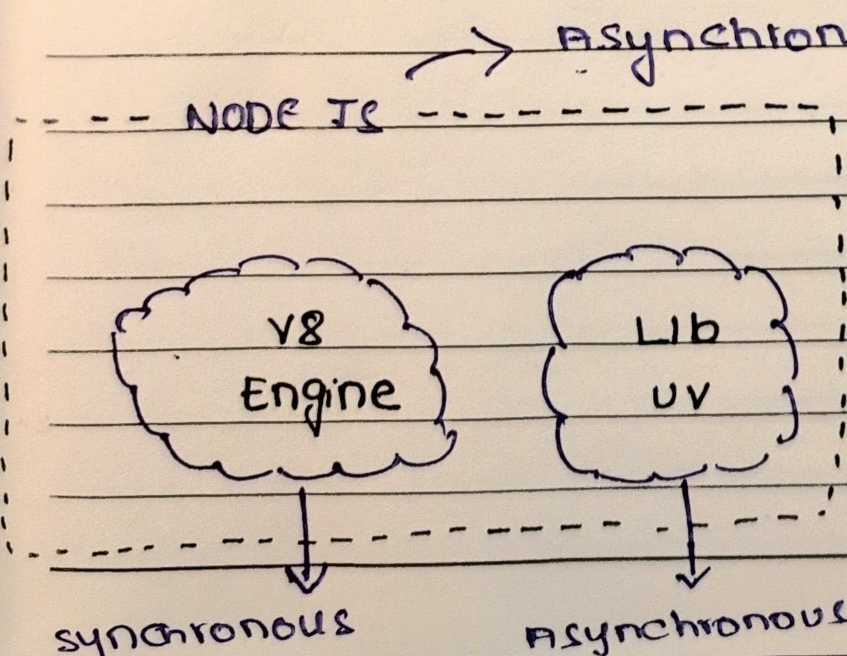
Asynchronous I/O
I/O = Input/output

For example:

```
var a = 10;
var b= 20;
function multiply (x,y) {
    const result = a*b
    return result ;
}
var c = multiply (a,b);
console.log (c);
```

It's a Non-Blocking thread

once the whole code is executed, the result will be the garbage collector. Because once it done what's the use of the result there & the whole code moves out of the call stack, [G.E.C] will be deleted.



* Node.Js is Asynchronous. Node.Js is mix of V8 engine(synchionous) & libuv (Asynchronous). So Node.JS is called "ASYNCHRONOUS I/O"

Date 12/02/2026

→ this is an async function and doesn't block the main thread.

"UTF-8" :: fs.readFile("./File.txt", "utf8", (err, data) => {
    console.log("File Data:", data);
});

UTF-8 :: Unicode Transformation Format - 8 bit

⇒ It's a translator that converts binary data into human readable text.

Without this "utf8" in this code the out put will be :

<Buffer 48, 65, 6C, 6C, 20, 66 72 ...>

which is completely binary format, but with that "utf8" the output will be ::

: whatever there in the file. [human readable text].

There will be functions like fs.readFileSync

(This is a synchronous function & blocks the main thread to execute this ⇒ will stop for sometime)

Date 12/02/2026

SETTIMEOUT: Since settime() is asynchronous handled by libuv & goes to the timer & callback to the V8 Engine and it will wait till the call stack to be empty and from the point the call stack is empty then it will print in the console by taking the timer.